MAD II Bootcamp – Session Plan

Oct 19, 2026 · @Aaron

Welcome

Over 10 days you will build one complete MAD II project from scratch: **Quiz Master**, a multi-user exam-preparation app. Days 1–7 are live-coding sessions of about 2 hours each. Days 8–10 are for clearing your doubts.

Each day adds one layer to the same app, so by Day 7 you will have a working project and understand every file in it.

Two kinds of users

- **Admin (Quiz Master)** – created automatically, no sign-up. Manages subjects, chapters, quizzes and questions, searches and blocks users, sees summary charts, exports reports.
- **User** – registers with email and password. Takes timed quizzes, sees scores and charts, exports their own scores as CSV.

Tech stack

Part	Tools
Backend API	Flask, Flask-SQLAlchemy, SQLite, Flask-Security-Too (token login)
Frontend	Vue 3 Options API (Vite), Vue Router, Vuex, Axios, Bootstrap 5, ApexCharts (vue3-apexcharts)
Background jobs	Celery + Redis (CSV export, daily reminders, monthly report emails)
Speed	Redis caching with Flask-Caching

Schedule at a glance

Day	Topic	By the end of the day you will have
1	Project setup & database	A Flask backend with all tables and an admin user created automatically

Day	Topic	By the end of the day you will have
2	Token-based login & roles	Register, login and logout APIs; admin-only and user-only access
3	REST APIs	Every feature working through the API
4	Vue 3, Router & Vuex	A Vue app with login, register and protected pages
5	Admin & user screens	The full UI: manage quizzes, take a timed quiz, see scores
6	Celery & Redis jobs	CSV export, daily reminder emails, monthly report emails
7	Caching, charts & final review	Faster APIs, summary charts, a project ready for submission
8	Doubt clearing 1	Backend and login doubts solved
9	Doubt clearing 2	Vue and Vuex doubts solved
10	Doubt clearing 3	Celery, Redis and caching doubts solved

Before Day 1: set up your laptop

Please finish this before the first session, so we can start coding right away.

Tool	Version	How to check
Python	3.10 or newer	`python --version`
Node.js (includes npm)	18 or newer	`node --version`
Redis (needed from Day 6)	6 or newer	`redis-cli ping` replies `PONG`
MailHog (optional, Day 6)	any	open http://localhost:8025
VS Code	any	–
DB Browser for SQLite (or a VS Code SQLite extension)	any	–

Installing Redis and MailHog:

- **macOS:** `brew install redis mailhog`, then `brew services start redis`

- **Ubuntu or WSL:** `sudo apt install redis-server`, then `sudo service redis-server start`
- **Windows:** install **WSL (Ubuntu)** and run Redis and Celery inside it. Celery does not run properly on native Windows.

Checklist:

- [ ] Python 3.10+ installed
- [ ] Node.js 18+ installed
- [ ] Redis installed and `redis-cli ping` gives `PONG`
- [ ] VS Code installed
- [ ] Windows only: WSL with Ubuntu installed

Day-by-day plan

Day 1 – Project setup & database

- **You will learn:** how MAD II differs from MAD I (Flask sends only JSON, Vue draws the pages), the Flask app factory, SQLAlchemy models and relationships.
- **You will build:** the project folders, `config.py`, all 7 tables (User, Role, Subject, Chapter, Quiz, Question, Score) and an admin user created by code.

Day 2 – Token-based login & roles

- **You will learn:** authentication vs authorization, token login, password hashing, 401 vs 403, role-based access, CORS.
- **You will build:** register, login, logout and "who am I" APIs with Flask-Security-Too; admin-only routes.

Day 3 – REST APIs

- **You will learn:** REST URL design, HTTP methods and status codes, validation, why scores are calculated on the server.
- **You will build:** create/read/update/delete APIs for subjects, chapters, quizzes and questions; quiz attempt and submit; user scores; admin search and block/unblock.

Day 4 – Vue 3, Router & Vuex

- **You will learn:** single-page apps, Vite project structure, Vue components with the Options API (data, computed, methods, watch, lifecycle hooks), Vue Router with guards, Vuex (state, getters, mutations, actions), Axios interceptors.

- **You will build:** login and register pages, one small Vuex store that remembers who is logged in (even after a refresh), admin and user pages that only the right role can open.

Day 5 – Admin & user screens

- **You will learn:** forms and tables with `v-model`, `v-for`, `v-if`; each page loading its own data when it opens; route and query params; watchers; a countdown timer.
- **You will build:** admin screens to manage subjects, chapters, quizzes, questions and users, with search; user screens to filter quizzes, take a timed quiz (auto-submits at 0) and see scores. Styled only with Bootstrap and mobile-friendly.

Day 6 – Background jobs with Celery & Redis

- **You will learn:** why slow work runs in the background, Celery worker vs Celery beat, Redis as the message queue, scheduled jobs.
- **You will build:** an "Export as CSV" button (user and admin), a daily reminder email at 6 PM, and a monthly activity report email on the 1st of each month.

Day 7 – Caching, charts & final review

- **You will learn:** caching with expiry and clearing the cache after changes, drawing charts with ApexCharts, a security checklist.
- **You will build:** Redis caching for the busiest APIs, summary charts for admin and user, and a full end-to-end run of your app.

Doubt-clearing days (Days 8–10)

No new topics on these days. Send your doubts **the day before** each session so similar questions can be grouped and answered together. Bring your own project open on your laptop – we debug live with screen sharing.

Day	Focus	Typical doubts
8	Backend & login	401/403 errors, token not working, model and relationship errors, CORS errors
9	Frontend	Data not updating on screen, Vuex flow, route guard loops, token not sent, Bootstrap layout on mobile
10	Jobs, caching & submission	Celery worker or beat not running, Redis errors, Windows/WSL issues, old data from cache

Running the finished app (Day 7)

Open 5 terminals:

```
# 1. Redis (skip if it already runs as a service)
redis-server

# 2. Backend API  (inside quiz-master/backend)
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python seed_demo.py      # optional demo data, run once
python app.py            # http://localhost:5000

# 3. Celery worker (backend folder, venv active)
celery -A worker.celery worker --loglevel=info

# 4. Celery beat (backend folder, venv active)
celery -A worker.celery beat --loglevel=info

# 5. Frontend  (inside quiz-master/frontend)
npm install
npm run dev              # http://localhost:5173
```

Demo logins: admin `admin@quizmaster.com` / `admin123`, user `user@quizmaster.com` / `user123` (after `seed_demo.py`).

Tips: code along in every session, commit your code at the end of each day – each day builds on the last.